AKADEMIA TARNOWSKA

KATEDRA INFORMATYKI

Bezpieczeństwo i jakość oprogramowania

Laboratorium 2
Strumienie, interfejsy funkcyjne, wyrażenia lambda, optional.

```java
// 1. Functional interface
@FunctionalInterface  1 usage
interface MovieCondition {
    boolean test(String title);  1 usage
}

public class Demo {
    public static void main(String[] args) {
        List<String> movies = List.of("System", "Matrix", "Ex Machina", "Tron");

        // 2. Lambda
        MovieCondition startsWithM =  String title -> title.startsWith("M");

        // 3. Stream
        Optional<String> foundMovie = movies.stream()
                .filter(startsWithM::test)
                .map(String::toUpperCase)
                .findFirst();

        // 4. Optional
        String result = foundMovie.orElse( other: "Brak pasującego filmu");
    }
}
```

Tomasz Gądek

Cele laboratorium

Student potrafi:

- modyfikować bazę danych, backend i frontend pod nowe wymagania,
- obsługiwać żądania **REST API** oraz odpowiednie kody statusów **HTTP**,
- tworzyć własne interfejsy funkcyjne (**@FunctionalInterface**),
- zastępować klasyczne pętle strumieniami (**Java Stream API**),
- używać klasy **Optional** do bezpiecznej obsługi brakujących danych,
- osadzać zewnętrzne zasoby multimedialne w aplikacji.

Zarys sytuacji

Klient jest zadowolony z Twojej implementacji systemu, ale nie do końca spełnia jego oczekiwania. Jest skłonny zainwestować w projekt, ale będziesz musiał (musiała) dopisać kluczowe funkcjonalności.

Zadanie 1

Pobierz projekt **movies_part_02**:
https://drive.google.com/file/d/1gnQ-0qkqPhLQAqhU32kkxRILd0rRAGc_/view?usp=drive_link

Zadanie 2

Lista filmów powinna być posortowana malejąco po roku produkcji.

Zadanie 3

Klient zgłosił, że nie ma możliwości obejrzeć trailerów prezentowanych filmów. Z firmy outsourcingowej został pozyskany frontend developer, który wykonał za Ciebie część implementacji. Przejście do szczegółów filmu odbywa się po kliknięciu w jego obrazek na liście. Aby funkcjonalność działała w pełni zostałeś poproszony (poproszona) o dokończenie implementacji po stronie serwera. Programista frontendowy zostawił nam pewne wskazówki, które pomogą Nam w implementacji:

W **details-movie.html** zostanie użyta usługa typu **GET** o adresie:

http://127.0.0.1:8080/api/movies/{movieId}

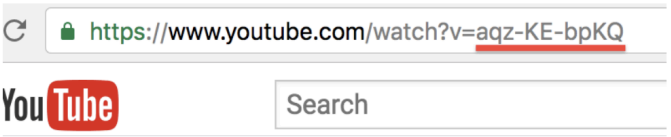
Usługa powinna zwracać **JSONa** (poniżej znajduje się przykładowy obiekt):

```
{
    "title": "Kontrola absolutna",
    "videoId": "_W_1dmcSthE"
}
```

Twoim zadaniem będzie dodanie kolumny **video_id** w tabeli **movies** oraz odpowiednie jej obsłużenie w każdej warstwie systemu. Doprowadź do tego aby usługa działała prawidłowo.

Programista pozostawił po sobie dokumentację, z której korzystał w celu osadzenia trailerów na Naszej stronie:
https://developers.google.com/youtube/player_parameters?hl=pl

Doczytaj w jaki sposób można uzyskać dostęp do **VIDEO_ID**, a następnie uzupełnij brakujące dane w tabeli.



Lista trailerów:

https://www.youtube.com/watch?v=46qKHq7REI4
https://www.youtube.com/watch?v=lEyrivrjAuU
https://www.youtube.com/watch?v=_W_1dmcSthE
https://www.youtube.com/watch?v=yG4omp0xO6o
https://www.youtube.com/watch?v=OD_vO9-Ztr8
https://www.youtube.com/watch?v=Rn2cf_wJ4f4
https://www.youtube.com/watch?v=3efV2wqEjEY
https://www.youtube.com/watch?v=L9szn1QQfas
https://www.youtube.com/watch?v=s0f3JeDVeEo

Zadanie 4

Po Twoich zmianach funkcjonalność dodawania filmów nie obsługuje trailerów. Dokonaj odpowiednich poprawek, aby umożliwić ich dodawanie (frontend + backend).

Zadanie 5

Spójrz na listę filmów. Usuwanie filmów jest nieintuicyjne. Użytkownik, który widzi taką listę nie będzie w stanie się domyślić w jaki sposób wywołać taką funkcjonalność.



System
ROK PRODUKCJI
1995
ID: 1

Zamień tekst na przycisku **ID: [id filmu]** na **Usuń film**. Operacje wykonaj dla wszystkich filmów dostępnych na liście.

Zadanie 6

Interfejs funkcyjny jest to interfejs, który posiada tylko jedną metodę abstrakcyjną. Opcjonalnie oznaczamy go adnotacją **@FunctionalInterface**.

Twoim zadaniem będzie refaktoryzacja wszystkich mapperów w aplikacji. Utwórz interfejs o nazwie **Converter**. Każdy mapper powinien implementować nowo utworzony interfejs. Przeanalizuj poniższy kod, a następnie postaraj się go wdrożyć w aplikacji.

```
class User { }
class UserDto { }

@FunctionalInterface
interface Converter<T, F> {
    T convert(F from);
}

class UserMapper implements Converter<User, UserDto> {

    @Override
    public User convert(UserDto from) {
        return new User();
    }
}

class UserDtoMapper implements Converter<UserDto, User> {

    @Override
    public UserDto convert(User from) {
        return new UserDto();
    }
}

public class Main {

    public static void main(String[] args) {

        final User user = new User();
        final UserDto userDto = new UserDto();

        // DTO -> ENTITY
        Converter<User, UserDto> userMapper = new UserMapper();
        User convertUser = userMapper.convert(userDto);

        // ENTITY -> DTO
        Converter<UserDto, User> userDtoMapper = new UserDtoMapper();
        UserDto convertUserDto = userDtoMapper.convert(user);
    }
}
```

Zadanie 7

Bardzo proszę przerobić pętle na strumienie (jedno z takich miejsc to np. mapper do listy filmów).

Zadanie 8

Optional w języku Java to obiektowy kontener reprezentujący wartość, która może istnieć lub być pusta. Służący do bezpiecznego zarządzania brakiem danych i zapobiega błędom **NullPointerException**.

Przed usunięciem filmu proszę sprawdzić (SELECT na bazie danych) czy film o przekazanym ID istnieje. Takie zapytanie powinno zwracać **Optional<Movie>**. Dopiero gdy taki film istnieje należy go usunąć z bazy danych. Jeżeli film nie istnieje usługa powinna zwracać kod błędu **404 NOT FOUND**.